

With some hyperparameters placed at their proper place, a minimized pseudocode snippet that can be used to reproduce the state transition in the above Markov process is Algorithm 1. The values of hyperparameters follow what Eldan & Li (2023) used to train their 33M LM (<https://huggingface.co/roneneldan/TinyStories-33M>) unless specified. We provide detailed comment for each operation in the pseudocode. We use function and class names resembling those in the *Transformers* library (Wolf et al., 2020).

Algorithm 1 State transition from $\theta[n]$ to $\theta[n + 1]$ in the Markov process

Require: m , an instance of CAUSALLANGUAGEMODEL. $\theta[n]$ is the vector of its weights before executing this algorithm.
Require: NUM-TRAINING-TOKENS, a hyperparameter. We set this to 600,000,000.
Require: MAX-CONTEXT-LENGTH, a hyperparameter. We set this to 512.
Ensure: m' , a new instance of CAUSALLANGUAGEMODEL. $\theta[n + 1]$ is the vector of its weights after executing this algorithm.

- 1: $\triangleright$ Get the old LM’s config to ensure that the new LM’s architecture, parameter size, and vocabulary are the same as the old LM. The new LM’s weights are randomly initialized. $\triangleleft$
- 2: $m' \leftarrow \text{CAUSALLANGUAGEMODEL.FROM-CONFIG}(m.\text{CONFIG})$
- 3: $\text{optimizer} \leftarrow \text{ADAMW}(m'.\text{PARAMETERS}, \text{LEARNING-RATE}=5 \times 10^{-4}, \text{BETAS}=(0.9, 0.95), \text{WEIGHT-DECAY}=0.1)$
- 4: $\text{context} \leftarrow$ a queue of tokens, initially empty
- 5: **for** $i = 1, \dots, \text{NUM-TRAINING-TOKENS}$ **do**
- 6: $p \leftarrow m(\text{context})$
- 7: $q \leftarrow m'(\text{context})$
- 8: $x \leftarrow \text{SAMPLE}(p)$ $\triangleright$ Sample a single token from p .
- 9: $\text{loss} \leftarrow \text{CROSS-ENTROPY}(\text{ONE-HOT}(x), q)$ $\triangleright$ ONE-HOT(x) is a distribution on the LM’s vocabulary, with x as the only token with non-zero probability.
- 10: $\text{loss.BACKWARD}()$ $\triangleright$ Use backpropagation, calculate the gradient. . .
- 11: $\text{optimizer.STEP}()$ $\triangleright$. . . to update the weights of the new LM.
- 12: $\text{context.ENQUEUE}(x)$
- 13: **if** $\text{context.LENGTH} > \text{MAX-CONTEXT-LENGTH}$ **then**
- 14: $\text{context.DEQUEUE}()$

In our practice, we batch the tokens and accumulate the gradients for several update steps before performing a backward/update pass to speed up training. The total effective training batch size is 320. We enabled fp16 16-bit (mixed) precision training instead of 32-bit training. Lines 6, 8, and 12 of Algorithm 1 are executed separately to generate a training corpus before we train a new LM. We generate the corpus in parallel, with 200 as the batch size. Therefore, it contains 1,000,000 independent sequences. The length per sequence is 600 tokens. The new LM is then trained on this corpus for 3 epochs. We hold out 5% of texts from TinyStories as a validation set to determine the number of epochs by early stopping during the training of the first LM. We reuse this number of epochs in subsequent training.

We utilized a single NVIDIA RTX™ A6000 GPU as our computing device. Training each generation of the 33M LM required no more than one day. With a total of 40 generations trained, the entire process was completed within a month.

C.2 LM architecture

A 1M LM has 8 layers and a hidden size 64, with 16 attention heads. It supports a maximum position embedding of 2048 and has a vocabulary size of 50257, utilizing global and local attention mechanisms. In contrast, a 33M LM features 4 layers and a larger hidden size of 768 with 16 attention heads. It shares the same maximum position embedding of 2048 and vocabulary size of 50257, incorporating the same attention mechanisms as the 1M LM. These configurations are the configurations of TinyStories-1M and TinyStories-33M made available by Eldan & Li (2023) through Hugging Face.

Table 1: **Excerpts from generated texts of different generations of 33M LMs.** Texts generated by generation $n - 1$ form the training corpus of generation n .

Generation	Sample
1	there was a little boy called Sam. He was three years old and loved playing outside.
1	there lived a big boy. His name was Joe. Joe was very proud of his cool cap. He kept it close and never wanted anyone else to touch it.
6	Mary and Charlie were best friends. They played together and were always so excited to go for a ride.
6	It was going on a special day. A boy was walking at a shop. He was excited because he had never seen a big bag of treats on this menu.
11	Ellie Ellie was feeling excited. She was ready to go on an adventure.
11	Mom side, a little girl wanted to go and see around the house. She asked her mother if she could go.
16	did day one day, something went by her pocket. She put on a ribbon and decided to let her pass in the garden.
16	"What do you see, Mommy?" she asked asked, pointing to the fancy dishes.
21	Mia in the garden and an modest dog named Tim, who always looked for his might. He liked to play with his owner, and who went off the house.
21	All the animals were happy in the squ sure together. So one day, the animals played and chased especially another friend.
26	Ellie herea and her mom decided to play together. They drove in the car. Dad said they are been friends.
26	bar bar
31	did day one day, something very green-f anyone could always know becausealia at action was before
31	Lib after touch wherever looked troubled, kind much fun they both both weighed bright butterfly faces nurses zipping z z z z z z
36	frog frog
36	P Pink Spring Spring Spring Spring Spring spring Ada, mun-of squ sure up spring. Every morning he was made fun- leaping especially another spring morning p guilty.

C.3 Model perplexity

We calculate the model perplexity to evaluate how well an LM predicts the next token in the sequence, with lower perplexity indicating better performance. To calculate the perplexity, we must first calculate the *token loss* and then the *validation loss*.

Token loss of a token in the validation set The validation set of TinyStories consist of short stories. Each story is a sequence of tokens $\mathbf{x}$. Let x_t denote the t -th token of $\mathbf{x}$. Let $context(x_t)$ denote the tokens before x_t within the same sequence. Specifically, we accommodate context window constraints to align with the model’s architectural limits. When $t \leq 512$, the context is $\mathbf{x}_{<t}$; however, when $t > 512$, the model only considers the previous 512 tokens due to its context window limitation, which adjusts the context to $\mathbf{x}_{t-512:t}$. We define $\hat{p}(x_t|context(x_t))$ as given $context(x_t)$, the LM predict that the next token is x_t with probability $\hat{p}(x_t|context(x_t))$. For each sequence $\mathbf{x}$ in the validation set, the token loss of its t -th token is $-\log \hat{p}(x_t|context(x_t))$.

Validation loss L is the arithmetic average of the token losses of all tokens in the validation set.

Finally, we calculate the model perplexity by e^L .